PLATEFORME NUMÉRIQUE PRÉFECTORALE

e-Servisy d'Ihosy

Présentation du projet

Synthèse du périmètre, des outils, des rôles et de l’architecture

1. Présentation générale

e-Servisy d'Ihosy est une application web qui facilite les échanges entre les citoyens et l’administration préfectorale. Elle réunit un espace citoyen pour préparer et suivre des démarches et un espace administratif pour instruire les dossiers, gérer les rendez-vous et mettre à disposition les documents associés.

Le projet vise à centraliser les informations, réduire les déplacements lorsque la démarche peut être préparée en ligne et rendre l’avancement des dossiers plus lisible. Les décisions administratives restent prises par les agents habilités.

2. Fonctionnalités principales

– Création et suivi de demandes administratives avec formulaires et pièces justificatives.

– Consultation des documents délivrés, notifications et informations utiles.

– Réservation et gestion de rendez-vous ; émission de tickets PDF.

– Dépôt et suivi de réclamations.

– Espace d’administration : instruction des dossiers, gestion des comptes et configuration de modèles ou de marques officielles.

Parcours type

Le citoyen se connecte, choisit une démarche, complète les informations et transmet les pièces. L’administration instruit le dossier et fait évoluer son statut (par exemple : en cours, complément demandé, approuvé ou rejeté). Le citoyen suit le traitement et télécharge le document disponible.

3. Rôles et responsabilités

Citoyen (CITIZEN) : crée et consulte ses propres demandes, pièces, rendez-vous, notifications et documents.

Administrateur (ADMIN) : traite les dossiers et réclamations, gère les rendez-vous et accède aux fonctions administratives autorisées.

Superadministrateur (SUPERADMIN) : dispose de droits supplémentaires de gestion, notamment la création de comptes administrateurs, selon les contrôles du serveur.

L'authentification et les rôles sont contrôlés côté API. Les données personnelles et les pièces d'un citoyen ne doivent pas être accessibles à un autre compte. Les autorisations exactes dépendent des routes et des règles métier implémentées.

4. Outils et technologies utilisés

Interface web : React 19, TypeScript, Vite, React Router, TanStack Query, React Hook Form et Zod.

API métier : Node.js, NestJS 11 et TypeScript ; les contrôleurs exposent les routes et les services appliquent les règles métier.

Données : PostgreSQL pour les données relationnelles et Prisma ORM pour l'accès depuis le backend.

Fichiers et PDF : stockage actuel dans le volume d'uploads du backend ; PDFKit et pdf-lib figurent parmi les outils de génération et de manipulation PDF.

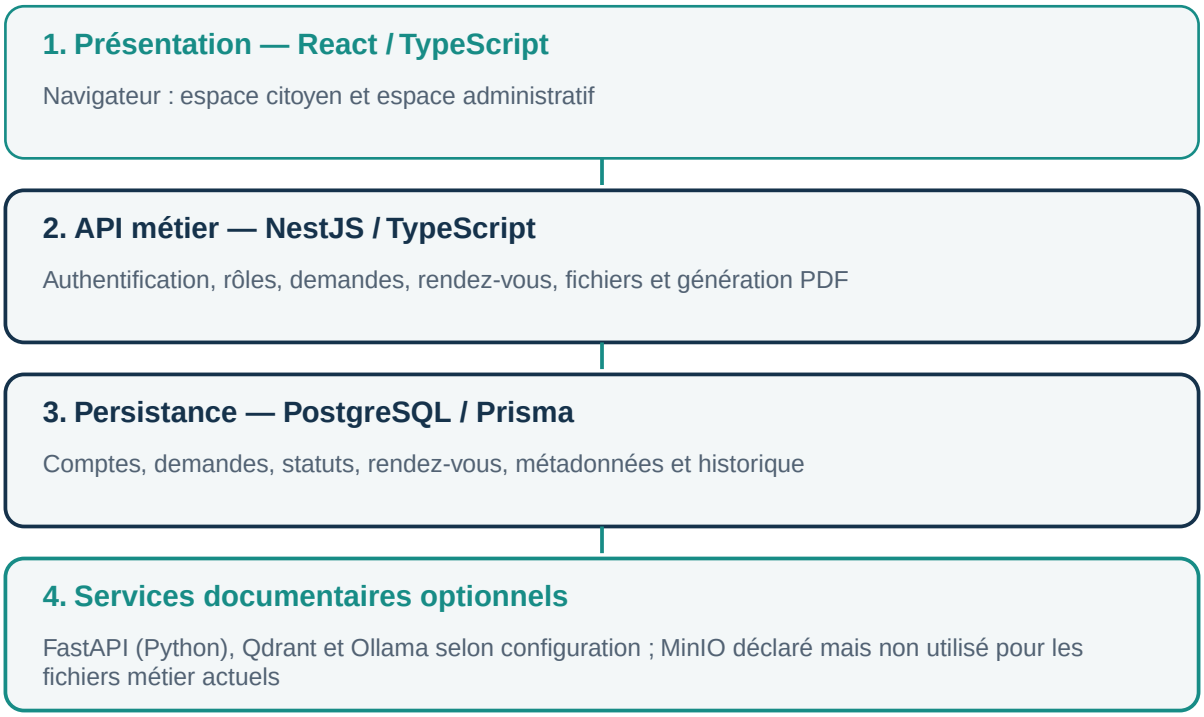
Environnement : Docker Compose pour lancer les applications et services ; npm pour les dépendances et scripts JavaScript/TypeScript.

Recherche documentaire : Qdrant est déclaré pour l'indexation vectorielle ; MinIO est déployé dans Compose, mais les pièces métier sont actuellement stockées dans le volume d'uploads.

Versions indiquées à partir des manifests du dépôt ; elles peuvent évoluer avec les mises à jour du projet.

5. Architecture de l’application

L’architecture sépare l’interface, le traitement métier, la persistance et les traitements documentaires. Le backend NestJS constitue le point de contrôle des identités, des droits et des règles de gestion.



Flux principal

– Le navigateur appelle l’API NestJS en HTTP/JSON et transmet les pièces en multipart.

– NestJS vérifie la session et les droits, applique les règles métier et lit/écrit les données via Prisma dans PostgreSQL.

– Les fichiers métier sont actuellement conservés dans le volume `uploads_data` ; l’API génère certains documents PDF.

– Les fonctions de recherche sémantique ou de traitement automatisé passent par les composants documentaires configurés, sans remplacer l’instruction humaine.

6. IA : périmètre et mode sans IA

Le portail de démarches, l'authentification, le suivi, les rendez-vous et les fonctions d'administration ne nécessitent pas l'IA pour leur usage principal. Le dépôt contient toutefois un service FastAPI séparé pour des traitements documentaires tels que l'OCR, les embeddings, la classification ou la génération, ainsi qu'une configuration Qdrant et Ollama.

Ces traitements sont conditionnels : Ollama n'est pas un conteneur Compose et doit être accessible sur la machine hôte ; les modèles doivent être installés. La documentation du projet précise que l'OCR et la classification ne doivent pas être présentés comme pleinement opérationnels sans installation, configuration et vérification dans l'environnement de démonstration. Une utilisation de l'application sans IA est donc possible pour le cœur des démarches, en laissant de côté les fonctions dépendantes de ces services.

7. Déploiement et points de vigilance

– Docker Compose déclare les conteneurs du frontend, du backend, de PostgreSQL, de Qdrant, de MinIO et du service FastAPI.

– Les volumes nommés assurent la persistance locale des données et fichiers ; les sauvegardes doivent être organisées et testées séparément.

– MinIO est présent dans l'infrastructure, mais son raccordement au stockage métier reste à réaliser pour qu'il serve effectivement aux pièces citoyennes.

– Le paiement documenté est un mode de démonstration ; ne pas le présenter comme une intégration effective d'opérateur sans vérification.

– Les identifiants de développement et ports d'infrastructure ne doivent pas être exposés tels quels en production.

En résumé

e-Servisy d'Ihosy est d'abord une plateforme web de démarches préfectorales avec des espaces distincts, des droits contrôlés et un suivi des dossiers. Son architecture modulaire permet d'ajouter des outils documentaires, mais le fonctionnement courant du portail ne dépend pas nécessairement de l'IA. Cette synthèse a été rédigée à partir des documents et manifests du dépôt, sans appel à un service d'IA externe.